FLYING SERVING: On-the-Fly Parallelism Switching for Large Language Model Serving

Shouwei Gao
Oregon State University
gaosho@oregonstate.edu

Junqi Yin
Oak Ridge National Laboratory
yinj@ornl.gov

Feiyi Wang
Oak Ridge National Laboratory
fwang2@ornl.gov

Wenqian Dong
Oregon State University
wenqian.dong@oregonstate.edu

arXiv:2602.22593v1 [cs.DC] 26 Feb 2026

Abstract

Production LLM serving must simultaneously deliver high throughput, low latency, and sufficient context capacity under non-stationary traffic and mixed request requirements. Data parallelism (DP) maximizes throughput by running independent replicas, while tensor parallelism (TP) reduces per-request latency and pools memory for long-context inference. However, existing serving stacks typically commit to a static parallelism configuration at deployment; adapting to bursts, priorities, or long-context requests is often disruptive and slow. We present *Flying Serving*, a vLLM-based system that enables online DP-TP switching without restarting engine workers. *Flying Serving* makes reconfiguration practical by virtualizing the state that would otherwise force data movement: (i) a zero-copy *Model Weights Manager* that exposes TP shard views on demand, (ii) a *KV Cache Adaptor* that preserves request KV state across DP/TP layouts, (iii) an eagerly initialized *Communicator Pool* to amortize collective setup, and (iv) a deadlock-free scheduler that coordinates safe transitions under execution skew. Across three popular LLMs and realistic serving scenarios, *Flying Serving* improves performance by up to 4.79× under high load and 3.47× under low load while supporting latency- and memory-driven requests.

1 Introduction

Large language models (LLMs) have become a primary workload in modern AI services, powering interactive assistants, code generation, and reasoning-heavy applications [1–6]. At production scale, *inference* often dominates both user experience and operating cost: providers must deliver responsive, streaming outputs under strict latency expectations while keeping expensive accelerator fleets highly utilized [7–9]. This is not a hypothetical concern. For example, Perplexity reports serving over 435M search queries per month, where each user query triggers multiple inference requests in the backend [10]. Large providers report similarly Internet-scale LLM demand: SageServe characterizes Microsoft Office 365 workloads with over 10M LLM requests per day across regions and time [11], while BurstGPT releases production traces from regional Azure OpenAI GPT services capturing burstiness and heterogeneity over long periods [12]. Cloud provider guidance similarly emphasizes that inefficient inference configurations can lead to "skyrocketing costs," motivating careful performance engineering and benchmarking [13, 14].

This operational reality is increasingly captured by benchmarking practice. MLPerf Inference evaluates *latency-bounded throughput*: reported throughput is only meaningful if the system satisfies explicit latency constraints under online arrivals [15–18]. For example, MLPerf specifies constraints in terms of time-to-first-token (TTFT) and time-per-output-token (TPOT), and recent releases explicitly introduce interactive, low-latency LLM benchmarks to reflect user-facing responsiveness requirements [12, 18–20]. More broadly, provider-facing benchmarking guidance frames deployment cost as a function of how many queries the system can serve per second *while remaining responsive* [21]. Together, these trends highlight a central systems objective for LLM serving: maximize throughput subject to end-to-end latency constraints, rather than optimizing either metric in isolation.

Current distributed inference stacks, however, force architects into a static choice between parallelism strategies. TP shards model weights across GPUs to reduce per-request latency and accommodate large models or long contexts under tight SLOs [22]. DP replicates the model to maximize aggregate throughput by serving independent requests concurrently. The problem is rigidity: real serving workloads are non-stationary, with bursty arrivals, heterogeneous request lengths, and mixed priority classes, so a single fixed strategy is inevitably suboptimal [23]. TP-centric deployments can under-provision concurrency during bursts and suffer queue buildup, while DP-centric deployments can inflate tail latency and struggle to sustain long-context sessions due to memory pressure from weights and KV cache. While other parallelization strategies, such as Pipeline Parallelism (PP) [24] and Expert Parallelism (EP) [25], are utilized in specific scenarios, particularly for Mixture-of-Experts (MoE) models [25, 26], DP and TP serve as the fundamental building blocks of distributed inference. Crucially, TP and DP are universally applicable and orthogonal to these specialized methods, allowing them to be compatible with other strategies as needed.

An ideal serving system should *adapt* its parallelism online. During low load, it coalesces devices into TP groups to minimize tail latency. During bursts, it should expand into DP replicas to increase concurrency and drain queues. Realizing this policy is technically challenging because DP and TP differ in (i) how they place and access weights, (ii) how they allocate and index KV cache, and (iii) how they form communication groups. Intuitive switching between modes requires weight replication or reload, KV cache migration or duplication, and disruptive reconfiguration of communication state, which costs that erase the benefit of adapting under load.

To address these challenges, we present *Flying Serving*, a distributed LLM serving system that dynamically switches between DP and TP *without* model replication, KV cache transfer, or prohibitive reconfiguration pauses. *Flying Serving* supports both execution modes from a unified in-memory organization: it (1) enables zero-redundancy weight management across DP/TP, (2) maintains a shared KV cache pool that remains stable across modes, and (3) activates the appropriate communication topology on demand. A workload-aware scheduler coordinates switching under execution skew and mixed priorities.

Specifically, we make the following contributions in this paper:

- We propose *Flying Serving*, a vLLM-based serving system that virtualizes each engine as a DP replica and uses a global task pool plus a workload-aware scheduler to dynamically *merge* replicas into TP groups (and *split* them back), enabling adaptation to bursty demand, mixed priorities, and long-context requests.
- We introduce a switching substrate that makes DP↔TP transitions lightweight: a *Model Weights Manager* that loads weights once and realizes TP via logical shard activation (no tensor movement), a *KV Cache Adaptor* that maintains a single physical KV pool with constant-time remapping across modes, and a *Communicator Pool* that activates the required control- and data-plane collectives on demand.
- We implement *Flying Serving* as a set of patches to vLLM [27], preserving core optimizations including PagedAttention [27], chunked prefill [28], and continuous batching [29].
- We evaluated *Flying Serving* under bursty, production-like workloads and achieved a speedup of 4.79x and 3.47x for high and low workloads, respectively.

2 Background and Motivation

2.1 Transformer Architecture

Transformer inference is organized around a small set of high-impact linear operators whose shapes and execution modes govern both compute intensity and memory movement. As shown in Figure 1, each Transformer block consists of an Attention(MHSA or GQA [30]) operator followed by a Feed-Forward Network (FFN) operator. Although conceptually simple, these operators exhibit different tensor shapes and data dependencies across *prefill*, *batched decoding*, and *single-token decoding*, which in turn drive the system-level behavior of LLM inference.

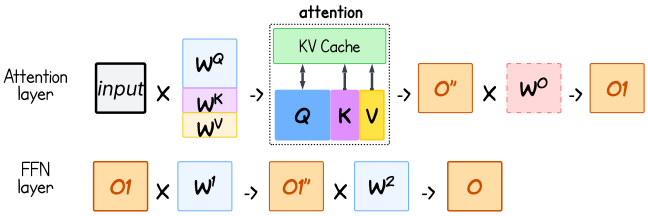


Figure 1: Attention/FFN operators (simplified).

Attention Layer. The MHSA sublayer is built from four projection matrices, i.e., Query (W_Q), Key (W_K), Value (W_V), and Output (W_O), that apply GEMMs to the input hidden state. In efficient implementations, the W_Q, W_K, and W_V projections are typically fused into a single GEMM operation applied to the input hidden state.

When the model processes a sequence of tokens, the fused projection generates corresponding q, k, and v vectors for each token in parallel. The resulting k and v vectors are appended to the KV cache, a persistent per-layer buffer. The attention mechanism then computes over the current and cached states to produce an intermediate output, which is finally projected by W_O to produce O^1.

FFN Layer. The FFN is composed of two large linear projections: an up-projection (W^1) and a down-projection (W^2), applied independently to each token. Despite their simplicity, these two GEMMs typically dominate FLOP count.

Crucially, this weight distribution dictates the topology of the KV cache. The KV cache consists of the intermediate key and value tensors generated by the W_K and W_V linear layers during the forward pass. Because these tensors are produced locally on the device holding the corresponding weight shard, the KV cache is inherently stored on the same device as the weights that generated it. Consequently, different parallelism configurations (e.g., varying the degree of TP) result in fundamentally different KV cache topologies and total capacities per device, creating a tight coupling between model weight distribution and memory management.

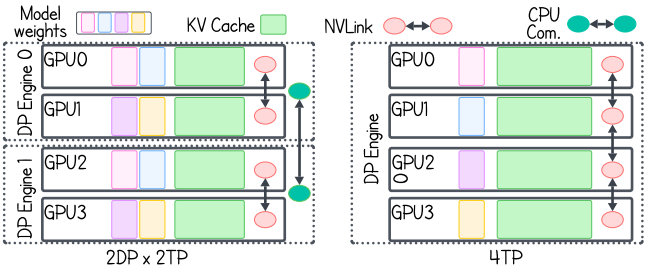


Figure 2: Model layouts on a 4-GPU node in DP vs. TP.

2.2 Data Parallelism and Tensor Parallelism

Serving large Transformer models requires distributing both computation and memory across multiple GPUs, and different workloads demand different forms of parallelism. DP and TP are two effective strategies for scaling large models and LLM serving. However, each strategy imposes its own requirements on weight layouts, KV cache organization, and communication patterns. These characteristics directly shape the design space explored in our framework design (Section 4): the feasibility of dynamic reconfiguration, the cost of state migration, and the limits on context length and throughput.

Figure 2 contrasts DP and TP on a 4-GPU cluster, and Figure 3 illustrates how our system leverages both modes by smoothly transitioning between them.

2.2.1 *Data Parallelism: Independent Engines with Replicated State.*

DP treats each group of GPUs as a standalone serving engine. In the 2DP × 2TP example in Figure 2, the cluster forms two identical engines: (GPU0, GPU1) and (GPU2, GPU3). Each engine holds replicated weight shards, runs its own inference loop, and serves disjoint request sets.

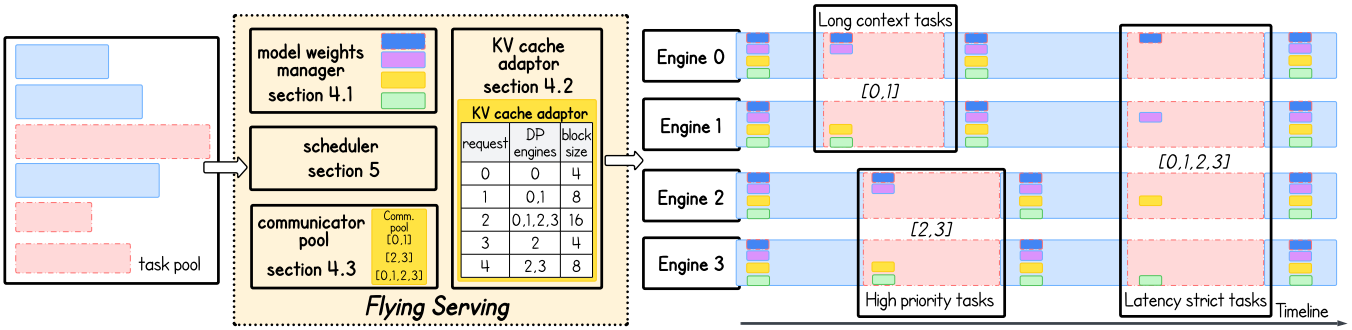


Figure 3: Overview of the *Flying Serving* architecture. The system functions as a middleware layer that orchestrates multiple engine workers, enabling dynamic transitions between DP and TP. The timeline on the right illustrates how the system adapts to different request types, such as high-priority, long-context, and latency-strict tasks, by reconfiguring workers from independent DP instances into cooperative TP groups on the fly.

(1) Memory duplication: Because each engine replicates the full model (or TP shard), the effective memory available for the KV cache shrinks.

(2) Minimal communication: Engines operate independently and synchronize only at coarse granularity (e.g., request scheduling or step alignment as in vLLM [27]), making DP suitable for low-overhead, high-throughput serving.

(3) Natural throughput scaling: Since each engine emits tokens independently, total tokens/sec increases linearly with the number of DP engines, even though individual requests are confined to the compute resources of one engine.

Together, DP excels for short-context or latency-sensitive traffic; however, it falls short for long-context or compute-heavy workloads that demand more memory or parallel compute than a single DP engine can provide.

2.2.2 *Tensor Parallelism: Distributed Operators for a Single Request.*

TP partitions each linear projection operator in both the attention and FFN layers, across multiple GPUs. In the pure 4TP configuration (Figure 2), all four GPUs jointly execute each operator invocation. This shifts the parallelization granularity from requests (DP) to operators (TP).

(1) Aggregated memory for weights and KV cache: Sharding weights across devices eliminates duplication. The KV cache inherits this sharding, effectively increasing the usable memory budget by (roughly) the TP degree. This offers longer context windows than DP.

(2) Lower latency for compute-bound workloads: Prefill and batched decoding benefit from parallel operator execution across GPUs, reducing per-request latency.

(3) High communication intensity: Every operator invocation requires synchronizing intermediate tensors among all TP workers. This mandates high-bandwidth interconnects (e.g., NVLink) and creates non-trivial *reconfiguration challenges* if TP groups are changed dynamically.

TP is necessary for long-context or high-compute requests, but expensive to maintain during heterogeneous workloads.

2.3 User Scenarios Motivating Dynamic DP-TP

Existing systems typically commit to a fixed parallelism strategy: either primarily DP or primarily TP. However, real serving workloads are heterogeneous in time, priority, and context length, and no single static configuration can serve all of these efficiently. We highlight three common scenarios where dynamically switching between DP and TP yields substantial benefits.

Use Case 1: Adapting to Time-Varying Load. Serving traffic is highly non-stationary. During low-load windows, the serving system backend has spare GPU capacity and can afford to over-provision resources per request. In this regime, forming TP groups is beneficial: the system aggregates GPUs to accelerate individual requests, lowering latency and improving the tail quality of service (QoS). When the workload spikes and request queues build up, the objective flips: the system should dissolve TP groups into more DP engines to maximize aggregate throughput and drain the queue. A dynamic DP-TP system can track load and continuously rebalance between "few fast TP engines" and "many DP engines" rather than committing to one extreme.

Use Case 2: Priority-Aware Service Differentiation. Multi-tenant deployments often enforce tiered service levels, where a subset of requests (e.g., premium users or critical applications) carry stricter latency SLOs. This is analogous to the scheduler's Quality of Service (QoS) mechanism in HPC facilities, where certain jobs are granted higher scheduling priority. Under a static configuration, either all requests benefit from TP (wasting resources on low-priority traffic), or all are constrained to DP (violating high-priority SLOs). With dynamic DP-TP, the scheduler can selectively assign high-priority requests to TP groups to minimize their latency, while routing best-effort traffic to DP engines that emphasize throughput. This enables fine-grained, per-request allocation of parallelism rather than a global, one-size-fits-all policy.

Use Case 3: Serving Long-Context Requests. The maximum context length a single serving engine can handle is bounded by its available GPU memory, dominated by the KV cache. In a pure DP configuration, if an engine supports sequences up to length L, a request with length $L + m$ will trigger an out-of-memory failure.

Dynamically merging multiple DP engines into a TP group effectively pools their memory: the KV cache is sharded across all participating GPUs, increasing the usable capacity by approximately the TP degree. This allows the system to "scale up" on demand to serve long-context requests that exceed the limits of any single engine, while retaining the ability to "scale out" via DP for shorter, throughput-oriented workloads.

3 An Overview of *Flying Serving*

Figure 3 shows *Flying Serving* as a middleware between a global *Task Pool* and a set of engine workers: in the static mode, engines run as independent DP instances; when a request requires tighter latency or more effective memory capacity, *Flying Serving* intelligently merges multiple engines into a TP group and later dissolves them back to DP.

Execution substrate: DP engines as the base unit. In *Flying Serving*, a single LLM engine is the fundamental *DP instance*. Each engine encapsulates one or more *Workers* mapped to one (or a fixed small set of) physical devices. Each worker maintains a persistent memory pool partitioned into (i) static storage for *Model Weights* and (ii) dynamic storage for the *KV Cache*. Incoming requests are first aggregated in a global *Task Pool*; by default, engines pull tasks and execute independently in DP mode to maximize throughput and absorb bursty arrivals.

One control abstraction: forming and releasing TP groups. The key capability of *Flying Serving* is to *bind* a subset of DP engines into a cooperative TP group for a specific request, then *release* those engines back to DP. This bind/release operation is the only switching primitive exposed to the scheduler; all complexity is contained in the middleware components that make switching cheap and safe at runtime:

- **Model Weights Manager (Section 4.1)** provides a rank-consistent *logical view* of weights under different TP degrees without requiring per-switch weight reloads.
- **KV Cache Adaptor (Section 4.2)** preserves a consistent physical KV allocation while remapping request-level KV layout across DP and TP execution, so request state remains valid through switches.
- **Communicator Pool (Section 4.3)** supplies ready-to-use communication groups for candidate engine sets, enabling immediate collective execution when engines are bound.
- **Scheduler (Section 5)** selects *when* to bind/release and *which* engines participate, ensuring deadlock-free coordination under execution skew.

How Flying Serving serves the three workload scenarios. The timeline in Figure 3 illustrates how the scheduler uses the same bind/release primitive to satisfy different request types. **(Use Case 1: workload pattern adaptation)** When load is high, *Flying Serving* keeps engines in DP to maximize concurrency and drain the queue; under light load, it opportunistically forms TP groups to reduce per-request latency. **(Use Case 2: priority-based differentiation)** High-priority tasks can trigger an immediate TP binding to obtain more compute per request and tighter latency, while normal tasks continue to execute on remaining DP engines. **(Use Case 3: long-context scaling)** Long-context tasks that pressure KV capacity are routed to wider TP groups so that KV is effectively sharded across engines; once the long-context request progresses past the memory-critical phase, engines are released back to DP to recover throughput.

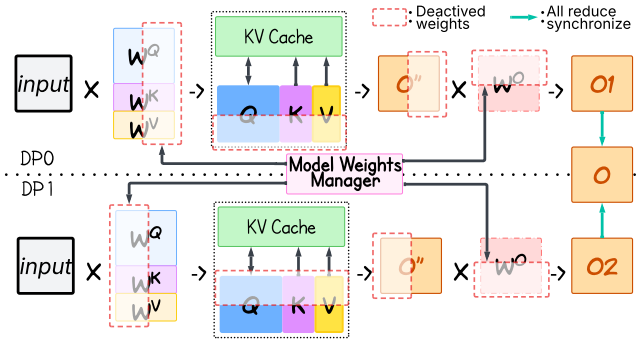


Figure 4: Model Weights Manager architecture for zero-copy DP/TP switching.

4 Runtime Substrate for Dynamic Parallelism

4.1 Model Weights Manager

Dynamic switching between DP and TP fundamentally changes how GPU workers consume model parameters. In DP, each engine requires a full replica of the model weights, whereas in TP, each engine accesses only a shard of each linear operator. Naively transitioning between these modes would require reallocating or reloading large parameter tensors, incurring transient memory and communication overhead. Such costs are incompatible with online serving and directly reduce the memory budget available for the KV cache.

The Model Weights Manager eliminates this reconfiguration bottleneck by decoupling *logical* weight sharding from *physical* weight storage. Its core invariant is that model parameters are loaded exactly once per engine and never physically moved thereafter. Mode switching is realized purely by changing which portions of an existing weight tensor are *activated* for computation.

In Transformer architectures, the majority of parameters reside in linear layers, specifically the QKV (W^Q, W^K, W^V) projections and the Output projection (W^O) in Attention layers, as well as the Up, Gate, and Down projections in FFN (as referred in Figure 1).

Figure 4 illustrates our partitioning strategy using an Attention layer as an example. In a standard DP configuration, both Engine 0 and Engine 1 hold identical, full replicas of the weights (W^Q, W^K, W^V and W_O). When the scheduler triggers a transition to a 2× TP configuration, the system must logically partition these weights across devices to parallelize computation.

4.1.1 Logical Weight Resharding. Specifically, we adopt the Megatron-LM style parallelism [22] to enabledynamic sharding:

- **Column-Parallel Layer (W^{QKV}):** We shard the fused W^{QKV} projection along its output (column) dimension. In Figure 4, Engine 0 activates the first column slice while Engine 1 activates the second; all other columns in each local replica are *deactivated* and excluded from computation. This partition maps cleanly to multi-head attention: each engine produces its local Q, K, and V

sub-tensors for a disjoint subset of heads, and thus requires no cross-engine communication for the W^{QKV} projection.

- **Row-Parallel Layer (W^O):** We shard the output projection W^O along its input (row) dimension. Each engine multiplies the full attention output with its local row slice, producing a partial projection (O_1 on Engine 0 and O_2 on Engine 1). A single *all-reduce* then aggregates these partial results to form the final output activation O for the layer.

This pattern ensures that only one synchronization step is required per pair of linear layers, maintaining high computational efficiency.

4.1.2 Zero-Copy View Implementation. A naive implementation of the above switching logic involves creating new shard tensors for each TP rank, which introduces duplicate data and causes memory allocation overhead. Instead, *Flying Serving* implements a non-invasive patch to vLLM's `linear.py`[1] module to enable *zero-copy*.

Specifically, when m DP engines merge into an m-way TP group, the Manager assigns each engine a unique rank ID $r \in [0, m-1]$, and creates a *view* of active weights on top of the corresponding full matrix:

$$W_{active}^{(r)} = \text{View}(W_{full}, \text{dim}, r, m) \tag{1}$$

where `dim` defines the sharding dimension (columns for W^{QKV}, rows for W^O).

This approach guarantees that the active weights used for TP execution are contiguous in virtual memory but map to the existing physical memory of the DP replica. By avoiding physical data movement or duplication, *Flying Serving* maximizes the available memory for the KV cache, distinguishing our approach from methods that rely on redundant storage or expensive reloading [23, 32].

4.2 KV Cache Adaptor

The KV Cache Adaptor enables mode switching without migrating KV state or rebuilding the cache allocator. The core difficulty is that TP changes the per-device KV tensor shape: under DP, each worker stores KV tensors for the full hidden dimension, while under TP, each worker stores only a $1/p$ slice for TP degree p. In existing engines, these shapes are baked into the allocator at initialization; switching DP↔TP would therefore require reallocating the KV pool and repopulating metadata, which is too costly on the serving critical path.

4.2.1 Challenge: Mode-Dependent KV Layouts. DP and TP induce different per-device KV tensor shapes. Let N be the number of cached tokens and D the model hidden dimension. In DP mode, each device processes independent requests and must store the full hidden dimension for every token. The memory requirement per token on a single device is proportional to D. In TP mode, the model weights and intermediate tensors are sharded across p devices. Consequently, the KV cache for a single token is partitioned, which means each device stores a slice of size D/p.

[1]vLLM implements its core linear operators (including tensor-parallel variants) in `vllm/model_executor/layers/linear.py` [31]. Our patch extends this operator to accept a rank-aware tensor *view* of the original weight matrix, allowing dynamic slicing along the sharding dimension without allocating new storage or modifying kernel code.

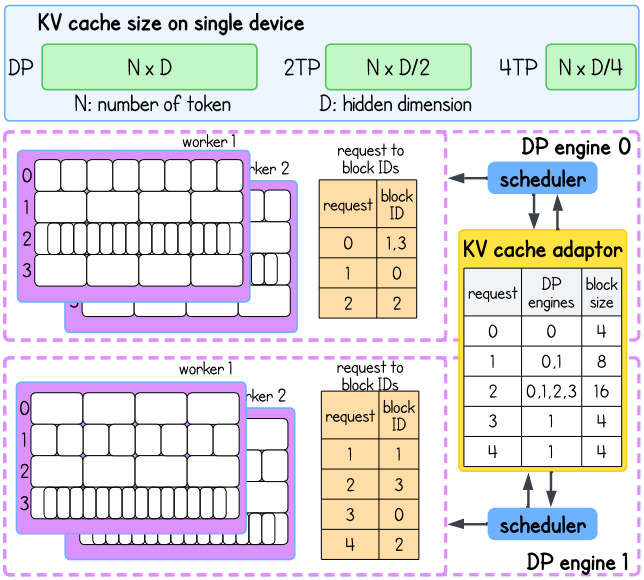


Figure 5: An example of KV cache adaptation in *Flying Serving*. With per-token footprint shrinks with TP (DP: $N{\times}D$; 4TP: $N{\times}D/4$), to keep a fixed physical layout without reallocation, we scale block size inversely with TP: 4 tokens (DP), 8 (2TP), 16 (4TP), managed per request by the KV cache adaptor; physical memory of each block is unchanged.

In existing serving engines, the KV allocator is initialized with a fixed block shape (determined by D and a pre-set block size) derived from a single deployment configuration. Switching from DP to TP reduces the local tensor width from D to D/p. If the system retains the original allocator settings, memory becomes misaligned and fragmented. Reconfiguring the allocator to match the new shape requires rebuilding the memory pool. Such an operation is orders of magnitude slower than the per-step execution budget and would stall latency-critical requests.

4.2.2 Unified Memory via Adaptive Block Sizing. To address this, *Flying Serving* introduces the *KV Cache Adaptor*, a middleware layer that dynamically manages the KV cache block. In vLLM's PagedAttention design, the KV cache is allocated as non-contiguous fixed-size physical blocks on the GPU and indexed via an OS-style logical mapping table. Instead of resizing physical memory blocks to fit the changing tensor shapes, we leverage the inverse relationship between the local hidden dimension D and the token capacity per block B.

Concretely, the bytes consumed by a physical block are:

$$M_{\text{block}} = B \cdot D_{\text{local}} \cdot P_{\text{size}}, \tag{2}$$

where B is the number of tokens per block, D_{local} is the per-device hidden dimension under the current mode, and P_{size} is the element size. Our key insight is to keep M_{block} constant across all modes to avoid reallocation. We achieve this by inversely scaling the block size B against the TP degree p. Formally, for a given parallelism degree p, the local dimension becomes $D_{\text{local}}(p) = D/p$. To maintain

memory alignment, we adjust the block size $B(p)$ such that:

$$B(p) = p \cdot B_{\text{base}} \tag{3}$$

Where B_{base} is the block size configured for the DP mode.

At runtime, as illustrated in Figure 5, the scheduler issues allocation and append requests in request space (e.g., request IDs), while the *KV Cache Adaptor* resolves these requests to physical block IDs in the shared table. In DP mode, each DP engine maintains independent request streams; therefore, the adaptor can map a request to one or more block IDs that may be local to a specific engine (e.g., request $0 \mapsto \{1, 3\}$, request $1 \mapsto \{0\}$, request $2 \mapsto \{2\}$ in the example table). When the system switches to TP, the adaptor preserves the physical residency of existing blocks and only updates the logical interpretation of each block (i.e., the effective block capacity $B(p)$) and the corresponding request-to-block mapping. In this way, mode transitions require neither KV-state migration nor allocator reinitialization: the adaptor performs constant-time metadata updates (capacity and indirection tables), while the underlying block pool and block IDs remain stable across DP/TP configurations.

4.2.3 Implementation and Benefits. We implemented this logic via a lightweight patch to the vLLM KV cache manager [2]. The adaptor intercepts requests and dynamically updates a *Logical Table* with the correct block size corresponding to the current parallelism mode of the worker. The worker informs both the KV cache kernel and the attention kernel of the stride and capacity for the specific request, ensuring correct memory indexing.

This design cleanly separates logical KV addressing from the physical block pool. A mode switch therefore reduces to metadata re-interpretation, i.e., updating per-request strides and block semantics, while leaving the allocator and underlying memory region unchanged. In effect, switching avoids KV migration and pool reinitialization, preserving cache residency and making reconfiguration lightweight. Moreover, when we consolidate multiple DP replicas into a TP group, each request can draw from the combined KV budget: the higher tokens-per-block density increases effective KV capacity, enabling longer contexts or larger batches without giving up the throughput benefits of the remaining DP replicas.

4.3 Communicator Pool

Dynamic DP-TP switching requires changing which workers communicate and how. Existing serving stacks (including the vLLM infrastructure) typically construct a *single, static* communication topology at startup, where CPU collectives (e.g., Gloo [33]) for control and GPU collectives (e.g., NCCL [34] over NVLink/PCIe) for tensor synchronization. Recreating or mutating these process groups at runtime is both slow (often tens of seconds) and brittle: in a multi-threaded inference loop, mismatched group membership can easily trigger collective hangs or deadlocks.

To address this, *Flying Serving* introduces a *Communicator Pool* that decouples the initialization of communication channels from their execution. We abstract the communication architecture into

[2]In vLLM, KV cache allocation and block-table management are implemented in the PagedAttention subsystem (e.g., `vllm/v1/core/kv_cache_manager.py` and the associated cache manager). Our patch extends this logic to allow per-request block sizing without modifying the underlying physical KV memory pool.

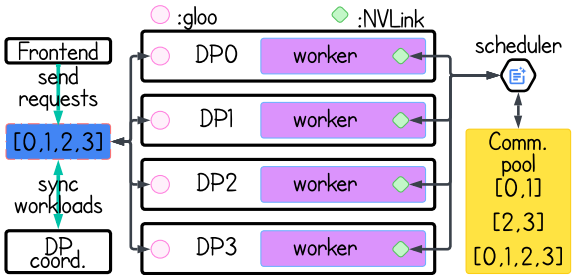


Figure 6: **Two-plane communication in *Flying Serving*. Left: Control plane (CPU–CPU).** The frontend routes requests and exchanges mode-switch signals with all DP engines via Gloo. **Right: Data plane (GPU–GPU).** The Communicator Pool pre-initializes all valid TP process groups (e.g., 2-way pairs and 4-way quartets) using NCCL. The scheduler activates the required group on demand, avoiding communicator creation on the request's critical path.

two distinct planes: the Control Plane (CPU-CPU) and the Data Plane (GPU-GPU).

4.3.1 Control Plane (CPU-CPU). The *control plane* provides request distribution and global state synchronization. Under DP, each request is assigned to a single engine; under TP, a single request must be delivered to multiple engines that temporarily operate as one TP group. As shown in Figure 6 (left panel), we utilize the existing CPU-CPU communication pipes (implemented via Gloo) not only for distributing requests but also for synchronizing system state. In the default design, DP engines periodically synchronize their execution state via an all-reduce to detect globally unfinished requests[3]. We use a *DP coordinator* piggybacks mode-switch signals (e.g., "`merge DP0 and DP1 into 2TP`") onto periodic synchronization heartbeats, ensuring that all participating engines observe the same transition point and apply it atomically.

4.3.2 Data Plane (GPU-GPU) with Eager Initialization. The dominant cost in reconfiguration is the GPU communicator setup (i.e., NCCL communication over NVLink or PCIe). Creating NCCL process groups and establishing connections can take seconds, which is unacceptable for mission-critical online serving.

To enable dynamic switching without the latency of runtime initialization, *Flying Serving* pre-allocates necessary communication groups at startup. However, naively enumerating all possible device combinations would lead to exponential resource overhead and memory exhaustion. Instead, we employ a *topology-aware initialization* strategy that strictly targets physically contiguous device groups. We build the pool in two steps:

(1) **Topology-Aware Group Identification:** Given N total DP engines (each using one GPU) and a set of supported TP degrees $\mathcal{P}$ (typically powers of two), we identify valid groups by partitioning the global rank space into contiguous segments. This constraint is critical because TP relies on high-bandwidth interconnects (e.g., NVLink), which typically connect adjacent GPU ranks. For example, with $N =$

[3]`vllm/v1/engine/core.py/#L1261`

4 DP and $\mathcal{P} = \{2, 4\}$, we do not generate strided or random combinations like $[0, 2]$ or $[1, 3]$. Instead, we restrict initialization to aligned, physically adjacent groups: $[0, 1]$ and $[2, 3]$ for 2TP, and $[0, 1, 2, 3]$ for 4TP. This reduction ensures that the number of communicators scales linearly rather than exponentially.

(2) **Pre-initialization:** For every identified group, we invoke `torch.distributed.new_group` (using the NCCL backend) during the system startup phase. The resulting communicator handles are cached in a hash map keyed by their member ranks (e.g., `Map<Tuple[int], Group>`). By pre-initializing only these topologically valid groups, *Flying Serving* ensures that when the scheduler decides to merge DP workers into a TP instance (e.g., merging workers 0 and 1), the required communicator is already active and can be retrieved in $O(1)$ time, avoiding the costly overhead of runtime group creation.

At runtime, switching modes reduces to selecting the appropriate pre-built communicator handle from the pool and routing collectives through it. No new process groups are created on the critical path, which avoids both initialization latency and the deadlock risk of on-the-fly group reconstruction. The memory overhead of keeping these pre-initialized communicators *inactive* is small: in our measurements, each PyTorch distributed process group consumes ~2 MB of host memory.

5 Dynamic Scheduler

The Dynamic Scheduler is the policy layer of *Flying Serving*: it decides *when* to form TP groups, *which* engines participate, and *how* to transition without stalling or deadlocking the serving loop. We implement the scheduler on top of vLLM v1[4] [35]. The scheduler's design is guided by two principles: (i) all engines participating in a TP step must observe an identical request order, and (ii) mode transitions must occur only at globally agreed safe points to avoid mismatched collectives.

5.1 The Base Execution Flow

The scheduler runs as a centralized event loop that coordinates K DP engines. Engines execute independently in DP mode by default; when a request is designated for TP, the scheduler temporarily coalesces a subset of engines into a TP group. Algorithm 1 summarizes one scheduling iteration, with six steps:

Step ❶ – Input Processing. At each iteration of the scheduling loop, the scheduler ingests newly arrived requests from the input socket and appends them to the local input queue Q_{in} by invoking `ProcessInputSocket()`. This step performs no scheduling or mode decisions and serves solely to collect pending requests for subsequent coordination.

Step ❷ – Global Synchronization. To ensure consistent task ordering and TP decisions across all engines, the scheduler synchronizes workloads with peers by invoking `SyncWorkload`(*peers*), which returns a globally agreed waiting queue Q_{wait}. All subsequent scheduling and mode-selection logic operates exclusively on Q_{wait}, guaranteeing that engines participating in a TP step observe the same request sequence.

Step ❸ – Resource Scheduling and Mode Determination. The scheduler initializes an empty worklist Q_{work}, clears the mode-switch flags $Flag_{SetTP}$ and $Flag_{ResetTP}$, and initializes the TP width variable N_{tp}. It then iterates over each request $req \in Q_{wait}$ to determine the execution mode. If $req.mode$ = TP, the scheduler sets N_{eng}, updates N_{tp}, and raises $Flag_{SetTP}$. Otherwise, it resets execution to DP mode by setting $N_{eng} = 1$ and raising $Flag_{ResetTP}$.

Step ❹ – KV Parameterization and Allocation. For each request, the scheduler derives mode-dependent KV cache parameters

Algorithm 1 Dynamic Scheduling & Mode Switching Protocol

Require: Base KV parameters: block size B_{base}; KV heads H_{base};
Start with all engines in DP mode ($N_{eng} \leftarrow 1$);
while True **do**
  ❶ Input Processing:
    $Q_{in} \leftarrow$ `ProcessInputSocket`();
  ❷ Global Synchronization:
    $Q_{wait} \leftarrow$ `SyncWorkload`(*peers*);
  ❸ Resource Scheduling:
    $Q_{work} \leftarrow \emptyset$;
    $Flag_{SetTP} \leftarrow False$;
    $Flag_{ResetTP} \leftarrow False$;
    $N_{tp} \leftarrow 1$;
  for $req \in Q_{wait}$ **do**
      ❸ Mode Determination:
      if $req.mode$ = TP **then**
        $N_{eng} \leftarrow req.num_engines$;
        $N_{tp} \leftarrow N_{eng}$;
        $Flag_{SetTP} \leftarrow True$;
      else
        $N_{eng} \leftarrow 1$;
        $Flag_{ResetTP} \leftarrow True$;
      end if
      ❹ KV Parameterization and Allocation:
      $B_{req} \leftarrow B_{base} \cdot N_{eng}$;
      $H_{req} \leftarrow H_{base}/N_{eng}$;
      $Blocks \leftarrow$ `KVCacheMgr.Allocate`(req, B_{req}, H_{req});
      Append ($req.id, Blocks$) to Q_{work};
  end for
  ❺ Mode Signaling (Collective RPC):
  if $Flag_{SetTP} = True$ **then**
    `RpcBroadcast` ("set_TP_mode", args=N_{tp});
  else if $Flag_{ResetTP} = True$ **then**
    `RpcBroadcast`("reset_TP_mode");
  end if
  ❻ Model Execution:
  `RpcBroadcast`("execute_model", args=Q_{work});
  `PublishOutput`($Output$);
end while

[4]vLLM v1 (SoTA version) was introduced as an *alpha* release in January 2025 as a major re-architecture of vLLM's core engine, refactoring components such as the scheduler, KV cache manager, workers, sampler, and API server while reusing substantial parts of the v0 codebase (e.g., model implementations, GPU kernels, and the distributed control plane).

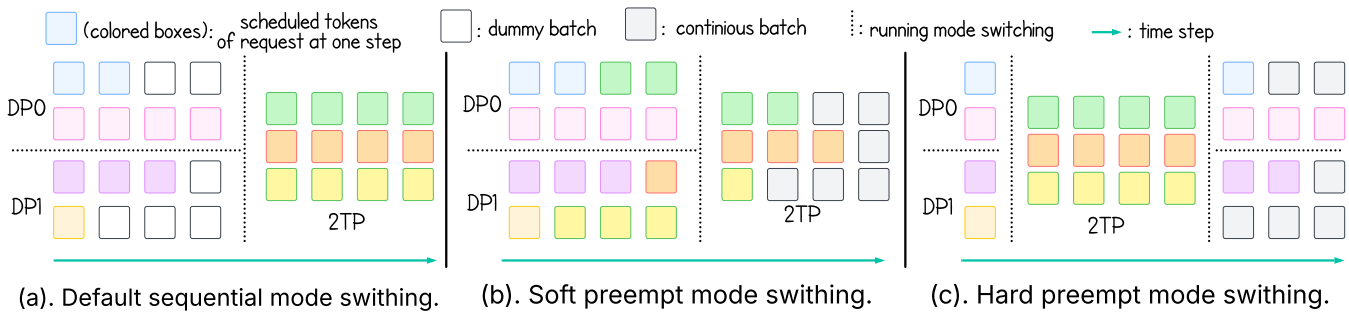


Figure 7: Comparison of Mode Switching Strategies in *Flying Serving*. (a) **Default Sequential Switching:** All workers must idle until the longest-running DP request completes, leading to resource inefficiency. (b) **Soft Preempt:** Workers with available slots pre-execute TP requests in DP mode while waiting. Although this requires recomputing the KV cache upon switching to ensure correct layout, it saves significant decoding time. (c) **Hard Preempt:** High-priority TP tasks immediately interrupt active DP requests. DP tasks are resumed later without recomputation, leveraging the unified KV Cache Adaptor.

based on the selected engine width:

$$B_{req} = B_{base} \times N_{eng}, \tag{4a}$$

$$H_{req} = H_{base}/N_{eng}. \tag{4b}$$

Using these parameters, it invokes `KVCacheMgr.Allocate` to allocate KV cache blocks and appends the resulting tuple $(req.id, Blocks)$ to the worklist Q_{work}.

Step ❺ – Mode Switching via Collective RPC. After processing all requests in the current iteration, the scheduler applies any required mode transition using collective RPCs. If $Flag_{SetTP}$ is true, it broadcasts `"set_TP_mode"` with argument N_{tp}; otherwise, if $Flag_{ResetTP}$ is true, it broadcasts `"reset_TP_mode"`. These broadcasts atomically configure the active communicator and execution mode across all engines before model execution.

Step ❻ – Model Execution and Result Publication. With execution modes and KV cache allocations finalized, the scheduler invokes `RpcBroadcast("execute_model", args=`Q_{work}`)` to execute one inference step collectively across engines. The generated outputs are then published via `PublishOutput()`, and the scheduler proceeds to the next iteration of the loop.

5.2 Adaptive Mode Switching Strategies

A central challenge in dynamic DP–TP execution arises from the service objectives of different user scenarios (Section 2.3). Real-world serving workloads are non-stationary (Use Case 1), contain a mix of latency-critical and best-effort requests (Use Case 2), and occasionally demand transient scaling of memory capacity for long-context inference (Use Case 3). In all three cases, the scheduler must react to mode-switch opportunities under execution skew: different DP engines reach scheduling boundaries at different times due to heterogeneous request lengths.

5.2.1 Default Sequential Switching. Figure 7(a) illustrates the baseline approach. When a TP request arrives, all DP engines wait for the longest-running request (the straggler) to complete before switching modes. While this strategy preserves correctness, it conflicts with the goals of Use Case 1 and Use Case 2: idle engines waste available capacity during load fluctuations, and latency-sensitive requests may be delayed by unrelated best-effort traffic. As a result, naive switching under-utilizes GPUs and introduces tail latency in practice.

5.2.2 Soft Preempt (Throughput-Oriented). Soft preempt addresses scenarios where throughput and load adaptability are the primary objectives (as in Use Case 1 and parts of Use Case 3). When a TP request is pending but some engines are still occupied with DP workloads, idle engines are allowed to speculatively execute the TP request in DP mode (Figure 7(b)). This leverages otherwise unused compute cycles instead of blocking on stragglers.

Although speculative execution produces a KV cache layout incompatible with TP, the trade-off is favorable. Decoding is typically memory-bound, while recomputation under TP is compute-bound and parallelized across multiple GPUs. By advancing generation during the waiting period, Soft Preempt amortizes transition delays and improves aggregate throughput under bursty or imbalanced workloads. This makes it well-suited for dynamically rebalancing between "many DP engines" and "few fast TP engines" as load fluctuates.

5.2.3 Hard Preempt (Latency-Oriented). Hard Preempt targets scenarios where immediate service is required, aligning directly with priority-aware service differentiation in Use Case 2. When a high-priority TP request arrives, the scheduler interrupts all active DP execution across participating engines and executes the TP request without waiting for stragglers (Figure 7(c)).

Crucially, this interruption does not impose recomputation overhead on paused DP requests. Leveraging the unified KV Cache Adaptor (Section 4.2), KV blocks with different logical layouts can coexist in the same physical memory pool. As a result, interrupted DP workloads retain valid KV state and resume execution seamlessly after the TP request completes. Hard Preempt thus enforces low tail latency without sacrificing correctness or throughput for background traffic.

5.3 Discussion

5.3.1 Generality. While motivated by specific user scenarios, Soft and Hard Preempt represent general scheduling primitives rather than workload-specific heuristics. Soft Preempt captures a broad

class of speculative execution techniques that trade limited recomputation for improved utilization, while Hard Preempt generalizes preemptive scheduling to distributed TP execution without KV migration. Together, they allow the scheduler to continuously navigate the latency vs. throughput vs. memory trade-off space under dynamic conditions.

Importantly, both strategies extend the base execution flow without violating its core invariants: TP execution observes a globally consistent request order, and all mode transitions occur only at scheduler-coordinated safe points. This separation of *mechanism* (dynamic DP–TP switching) from *policy* (Soft vs. Hard Preempt) enables *Flying Serving* to support a wide range of serving objectives without redesigning the underlying system.

5.3.2 Limitations. While *Flying Serving*'s design focuses on improving throughput and latency flexibility, the current implementation is designed primarily for intra-node parallelism. Specifically, our system assumes that the model weights and the necessary KV cache can be managed within the memory hierarchy of a single multi-GPU node (e.g., 8× H200). Consequently, extremely large models that necessitate multi-node model parallelism are outside the current scope of this work. However, we note that the vast majority of popular open-source models (e.g., Llama-3-70B, GPT-oss-120B) fit comfortably within a single modern GPU node, allowing multiple DP instances to run concurrently. Extending our dynamic reconfiguration protocol to support inter-node communication remains a promising direction for future research.

6 Evaluation

6.1 Experimental Setup

6.1.1 Hardware. We conduct our experiments on the Defiant cluster at Oak Ridge National Laboratory (OLCF ACE testbed) [36]. The system consists of 20 HPE Cray XD220 CPU nodes along with 2 HPE Cray XD670 Nvidia H200 GPU nodes, and each node is equipped with 8× NVIDIA H200 GPUs. Each H200 GPU provides 141 GB of HBM3e (4.8 TB/s peak bandwidth) and 1,979 TFLOPS peak dense FP8 throughput. The 8 GPUs are interconnected via NVLink, providing 900 GB/s bi-directional bandwidth.

6.1.2 Software, LLM Serving Models and Baselines. We implement *Flying Serving* on top of vLLM v1. We evaluate *Flying Serving* on three representative models from today's widely used LLM families: LLama, GPT, and Nemotron, spanning dense, sparse, and long-context serving:

- **Llama-3 (70B, dense)** [37]: a large, dense Transformer baseline that stresses compute and all-reduce bandwidth without sparsity.
- **GPT-OSS (120B, MoE)** [26]: a sparse Mixture-of-Experts model that activates only a small subset of experts per token, stressing routing, load balance, and sparse execution.
- **Nemotron (8B, dense)** [38]: an ultra-long context model that can digest up to 4M tokens to stress KV cache capacity and memory pressure.

We compare against three baselines: (i) **Static DP**, (ii) **Static TP**, and (iii) **Shift-Parallelism** [39]. Shift-Parallelism is a SoTA baseline as a *production-deployed* vLLM-integrated system that enables seamless runtime switching between latency-optimal TP and throughput-oriented sequence parallelism (SP) by exploiting KV cache invariance. The prior results show that this design has a strictly better latency-throughput trade-off than static DP/TP across arrival rates (e.g., up to ∼50% higher throughput and ∼1.51× faster response, with lowest completion time over rates), making it the most direct comparison point for our dynamic DP-TP switching.

6.1.3 Datasets and Synthetic Workloads. We evaluate on three open-sourced datasets: ShareGPT [40], CodeActInstruct [41], and HumanEval [42], covering conversational chat-based assistance, code-centric instruction following, and program synthesis.

We developed a tool to generate synthetic workloads to mimic real-world user requests because publicly available LLM datasets provide request contents but not realistic, reproducible arrival-time traces. A synthesized workload lets us precisely vary burst intensity and duration to stress mode-switching behavior and make comparisons repeatable across systems. This design aligns with the evaluation method from [39]. Specifically, we synthesize (1) *request lengths* with prompts sampled uniformly span $[128, 4000]$ input tokens and $[64, 512]$ output tokens; (2) *traffic pattern* where the arrival rate alternates between low load (2–5 req/s) and high load bursts (10–30 req/s); (3) *volume* in which each iteration issues 4000 requests to capture steady-state behavior across multiple bursts.

6.1.4 Performance Metrics. We use standard streaming-inference metrics that quantify initial responsiveness and steady-state token generation [43]:

(i) **Time To First Token (TTFT):** latency from when a request arrives at the serving system to when the first output token is generated (including both queuing and prefill).

(ii) **Time Per Output Token (TPOT):** the per-request average *time-between-tokens* during decoding, measured over consecutive output tokens after the first (i.e., inter-token interval).

(iii) **Peak generation throughput:** maximum aggregate output token rate (tokens/s) sustained by the system under load.

(iv) **Queue time:** time from request admission to first scheduling, isolating scheduler delay from execution time [44].

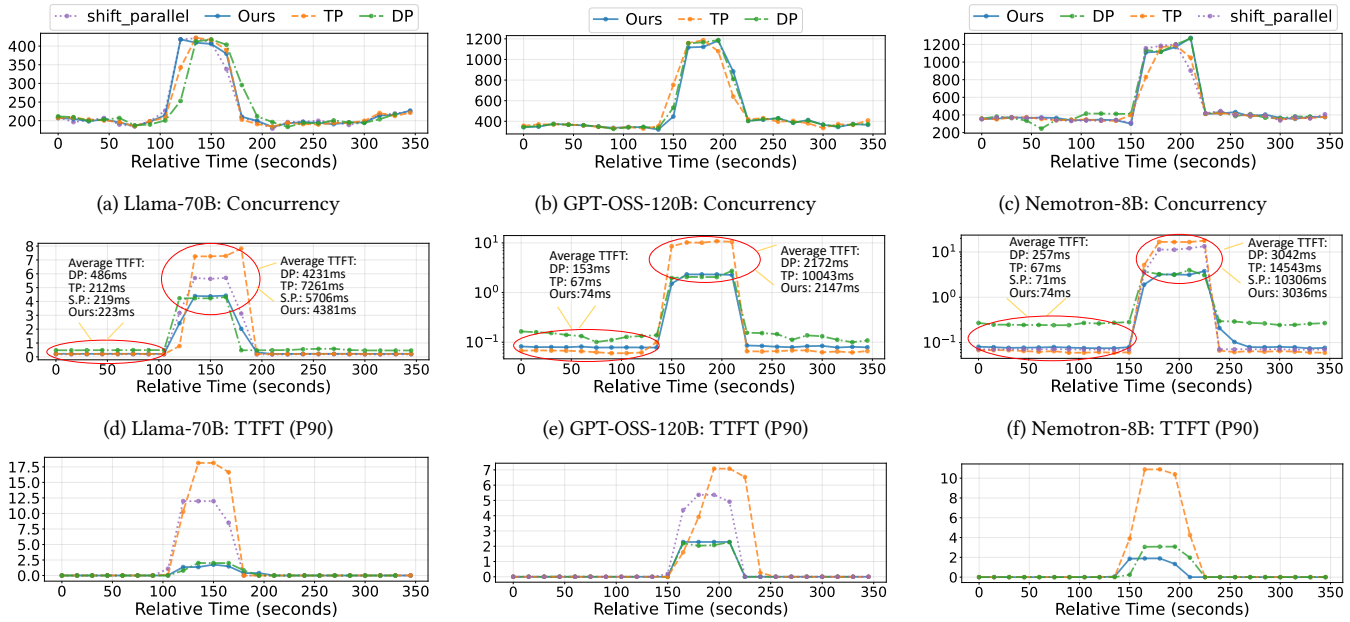
We export per-request and system metrics from the serving engine to Prometheus[45] and visualize time-series behavior in Grafana[46]. To minimize instrumentation overhead on the critical path, we log concurrency, TTFT, and queue time in the backend, and compute TPOT and aggregate throughput at the client from token timestamps.

6.2 Overall Performance

We compare *Flying Serving* against static TP, static DP, and Shift-Parallelism using the bursty workload from Section 6.1[5]. Figure 8 plots (top) in-flight concurrency, (middle) P90 TTFT, and (bottom) queue time over the trace; Figure 9 summarizes steady-state TPOT and peak generation throughput.

Same offered load. All systems observe the same concurrency pattern (Figure 8(a–c)), ensuring the latency (Figure 8(d–f)) and queueing differences (Figure 8(g–i)) are due to execution strategy rather than input load.

[5]Shift-Parallelism does not yet support GPT-OSS-120B due to compatibility issues with the new MoE kernel, so we are currently unable to report its results on GPT-OSS.



(a) Llama-70B: Concurrency (b) GPT-OSS-120B: Concurrency (c) Nemotron-8B: Concurrency

(d) Llama-70B: TTFT (P90) (e) GPT-OSS-120B: TTFT (P90) (f) Nemotron-8B: TTFT (P90)

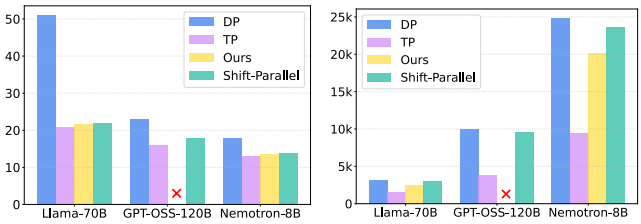
(g) Llama-70B: Queue Time (h) GPT-OSS-120B: Queue Time (i) Nemotron-8B: Queue Time

Figure 8: **End-to-end performance under bursty traffic.** Columns show Llama-3-70B (left), GPT-OSS-120B (middle), and Nemotron-8B (right). Rows report in-flight concurrency, P90 TTFT, and queue time. *Flying Serving* (blue) tracks load shifts: it avoids the TTFT/queue spikes of static TP (orange) during bursts, stays close to TP at low load, and substantially improves over static DP (green). Where supported, *Flying Serving* also outperforms the SoTA dynamic baseline Shift-Parallelism (purple).

Bursts: Flying Serving matches DP by avoiding queue build-up. During high-load phases, static TP (and Shift-Parallelism where applicable) accumulates substantial queueing (Figure 8(g–i)), which dominates TTFT (Figure 8(d–f)). In contrast, *Flying Serving* switches to DP and keeps queue time near DP, yielding TTFT close to the DP lower bound (e.g., average TTFT at burst: Llama-70B: 4.38s vs. 4.23s DP; GPT-OSS-120B: 2.15s vs. 2.17s DP; Nemotron-8B: 3.04s vs. 3.04s DP). Relative to static TP, *Flying Serving* reduces P90 TTFT by 1.66× (Llama-70B), 4.68× (GPT-OSS-120B), and 4.79× (Nemotron-8B). Relative to Shift-Parallelism, *Flying Serving* reduces P90 TTFT by 1.3× (Llama-70B) and 3.39× (Nemotron-8B).

Light loads (flat periods): Flying Serving stays near TP with small overhead. Under low load, *Flying Serving* remains in TP to minimize TTFT, closely tracking static TP (e.g., average TTFT: Llama-70B: 223ms vs. 212ms TP; GPT-OSS-120B: 74ms vs. 67ms TP; Nemotron-8B: 74ms vs. 67ms TP) while substantially improving over static DP. The remaining gap to TP reflects only mode-management overhead: 5.19% (12ms) for Llama-70B and 10.45% (7ms) for GPT-OSS-120B and 10.45% (7ms) for Nemotron-8B. In stead, Shift-Parallelism is up to ~4% faster at low load, but incurs significantly higher burst latency (3.39x than ours) and queueing (2.48x than ours).

TPOT and throughput: near-TP latency with near-DP capacity. Figure 9 shows that *Flying Serving* improves median TPOT over static DP by 2.31×, 1.28×, and 1.30× for Llama-70B, GPT-OSS-120B, and Nemotron-8B, respectively, approaching TP-like per-token latency. At the same time, *Flying Serving* retains ≈95–96% of DP peak throughput across models (e.g., 3,059 vs. 3,169 tokens/s on the largest size model Llama-70B), and outperforms static TP by 2.03×, 2.47×, and 2.52× in peak throughput. Where supported, *Flying Serving* also exceeds Shift-Parallelism in peak throughput (1.22× on Llama-70B; 1.18× on Nemotron-8B).



(a) Median TPOT (ms) (b) Peak Throughput (tokens/s)

Figure 9: Comparison of median TPOT and peak generation throughput across different models.

6.3 Performance on Workloads with Priority

We evaluate how *Flying Serving* handles mixed-priority serving on Llama-70B. The workload interleaves high-priority and normal requests, while the arrival rate is modulated between 3-5 requests/s to create sustained queueing pressure. We compare against two static baselines, i.e., static TP and static DP, under the same workload and report mean TPOT/TTFT (for priority requests and for all requests) plus peak throughput, following standard comparative methodology in systems evaluations. Table 1 summarizes the results.

Table 1: Llama-70B under mixed-priority workload.

Metric	static TP	static DP	Ours
Mean TPOT (priority) (ms)	22	32	24
Mean TPOT (all) (ms)	22	32	28
Mean TTFT (priority) (ms)	63	166	74
Mean TTFT (all) (ms)	2130	166	142
Peak Throughput (tokens/s)	2530	3164	3040

Priority requests. *Flying Serving* preserves near-TP latency for high-priority traffic: mean TPOT is 24 ms and mean TTFT is 74 ms, within 1.09× and 1.17× of the static TP baseline (22 ms / 63 ms). Relative to static DP, *Flying Serving* improves priority TPOT by 1.33× (32→24 ms) and TTFT by 2.24× (166→74 ms).

Overall system behavior under load. Static TP becomes throughput-limited and suffers severe queue time: mean TTFT (all requests) increases to 2130 ms. *Flying Serving* avoids this collapse by adapting parallelism to drain backlog, reducing mean TTFT (all) to 142 ms (15.0× lower than TP), and remaining 1.17× better than static DP (166 ms).

Throughput. *Flying Serving* sustains 3040 tokens/s peak throughput, retaining 96% of the DP baseline (3164 tokens/s) while delivering TP-like latency for priority requests.

6.4 Max Context Length and Switching Latency

We quantify *Flying Serving*'s ability to (i) expand KV cache capacity for long-context requests and (ii) reconfigure parallelism online. Unless otherwise noted, experiments run on 8× NVIDIA H200 GPUs hosting Llama-3-70B.

6.4.1 Context-Length Capacity. The maximum context length is bounded by the KV cache memory left after loading model weights. Table 2 shows that static deployments impose rigid limits tied to the fixed TP degree: 4DP×2TP can only support 264K tokens, and 2DP × 4TP only supports 959K tokens. By dynamically merging workers, *Flying Serving* scales KV capacity on demand and supports up to 1.9M tokens, a 7.2× increase over 4DP × 2TP, a 2.0× increase over 2DP×4TP, and within 17% of the static 1DP×8TP (the upper bound - "2.3M"). This result evaluates that *Flying Serving*'s reconfiguration support for dynamic DP-TP shifting introduces a small, fixed memory consumption; however, it frees up nearly all remaining GPU memory for the KV cache.

Table 2: Max context support and switching latency.

Configuration	GPUs/Inst.	Max Context	Switching Latency
Static 4DP × 2TP	2	264 K	292.38 s (Cold Start)
Static 2DP × 4TP	4	959 K	211.97 s (Cold Start)
Static 1DP × 8TP	8	2.3 M	146.54 s (Cold Start)
Flying Serving	dynamic	1.9 M	15 ms (Live)

6.4.2 Switching Latency. Static systems handle an over-limit request via a cold restart (shut down and re-launch with a higher TP degree), which requires reloading weights and re-initializing collectives; Table 2 shows these static methods taking non-negligible overhead, i.e., 146-292 seconds. In contrast, *Flying Serving* performs live switching using the pre-initialized *Communicator Pool* and zero-copy *Model Weights Manager* in Section 4, completing the switch in 15 ms, about five orders of magnitude faster (~10,000×), making dynamic DP-TP switching practical for large-scale, latency-critical serving.

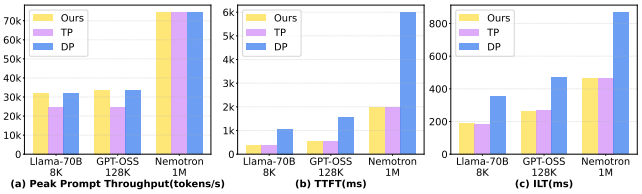


Figure 10: Comparison of Peak Prompt Throughput, TTFT, and ILT under long-contexts (8K, 128K, and 1M).

6.5 Ultra Long-Context Stress Test

We further stress the system at each model's maximum supported context length (8K for Llama-70B, 128K for GPT-OSS-120B, and 1M for Nemotron). Figure 10 reports peak prompt throughput, TTFT, and ILT[6]

Peak Prompt Throughput. Figure 10 (a) shows that *Flying Serving* sustains DP-level peak prompt throughput at the maximum context across all models. This translates to 1.29× (Llama-70B)and 1.38× (GPT-OSS-120B)higher throughput than static TP. For Nemotron-8B(1M), all three configurations converge at approximately 74K tokens/s, as the extremely long context relies more on memory bandwidth.

Latency Performance (TTFT and ILT). Figure 10 (b) shows our system achieves the TTFT performance nearly to the TP-like latency, with small overheads (< 1.1%). Also, compared to the static DP baseline, *Flying Serving* delivers substantial speedups of 2.94×, 2.78×, and 3.04× for Llama-70B, GPT-OSS-120B, and Nemotron-8B, respectively. Figure 10(c) shows that our system maintains TP-like inter-token latency, staying within 5% of static TP, while reducing ILT by 1.85 − 1.88× relative to static DP across all models.

Overall, *Flying Serving* removes the need to pre-commit to a single static configuration: it preserves DP-like capacity for long-context bursts while maintaining TP-like latency when load is light.

7 Related Works

Recent research in distributed LLM inference has shifted from static configurations to dynamic adaptability. We categorize these efforts below.

Dynamic Parallelism Transformation. Several systems enable runtime adaptability but face specific limitations. Hetis [47] uses a fine-grained and dynamic parallelism architecture that enables precise control over heterogeneous resources. LoongServe [48]. Gyges [49] accelerates state redistribution via weight padding yet still incurs latency penalties in the range of hundreds of milliseconds. Shift Parallelism [23] exploits KV cache invariance for seamless transitions between Tensor and Sequence Parallelism but is restricted to specific hardware topologies. Seesaw [29] dynamically

[6]Inter-Token Latency (ILT) measures the average time interval between the generation of consecutive tokens. We report ILT (instead of TPOT) because TPOT aggregates both compute and queueing/batching effects and can vary with scheduler decisions

re-shards models between prefill and decode phases; however, the overhead of frequent reconfiguration can negate throughput gains. Similarly, HAP [50] employs Integer Linear Programming to optimize hybrid strategies for MoE models, but the solver introduces significant runtime complexity.

Static and Scheduling-Based Optimization. Other approaches prioritize static planning or state abstraction. AlpaServe [51] uses compilation to generate optimal static plans, but cannot adapt running instances to traffic bursts without restarting. Tenplex [52] abstracts model state for dynamic training, yet lacks the sub-second responsiveness required for interactive inference. Finally, FastServe [32] optimizes throughput via preemptive scheduling and host memory offloading, but relies on limited PCIe bandwidth rather than flexible parallelism reconfiguration.

8 Conclusion

Flying Serving shows that static parallelism not be a deployment-time choice for LLM serving. By virtualizing weights, KV state, and communication, *Flying Serving* enables online DP↔TP reconfiguration as a lightweight operation coordinated by a deadlock-free scheduler. Across three models and realistic workloads, this capability translates into up to 4.79× speedup at high load and 3.47× at low load, while still accommodating priority and long-context requests.

9 Acknowledgment

This work is supported by the U.S. National Science Foundation (2514351 and 2505118). We thank the anonymous reviewers for their valuable feedback.

References

[1] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[2] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[4] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

[5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[6] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency, 2024.

[7] Samyam Rajbhandari, Mert Hidayetoglu, Aurick Qiao, Ye Wang, Juncheng Yang, Jeff Rasley, Michael Wyatt, and Yuxiong He. Arctic inference with shift parallelism: Fast and efficient open source inference system for enterprise ai. *arXiv preprint arXiv:2507.11830*, 2025.

[8] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of machine learning and systems*, 5:606–624, 2023.

[9] Ruiqi Lai, Hongrui Liu, Chengzhi Lu, Zonghao Liu, Siyu Cao, Siyang Shao, Yixin Zhang, Luo Mai, and Dmitrii Ustiugov. Tokenscale: Timely and accurate autoscaling for disaggregated llm serving with token velocity, 2025.

[10] Amr Elmeleegy, Lequn Chen, and Kevin Hu. Spotlight: Perplexity ai serves 400 million search queries a month using NVIDIA inference stack. https://developer.nvidia.com/blog/spotlight-perplexity-ai-serves-400-million-search-queries-a-month-using-nvidia-inference-stack/, December 2024.

[11] Shashwat Jaiswal, Kunal Jain, Yogesh Simmhan, Anjaly Parayil, Ankur Mallick, Rujia Wang, Renee St. Amant, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. Serving models, fast and slow: Optimizing heterogeneous LLM inferencing workloads at scale, 2025.

[12] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Burstgpt: A real-world workload dataset to optimize LLM serving systems, 2024.

[13] Shantanu Deepak Patankar and Hugo Affaticati. Optimizing language model inference on Azure. https://techcommunity.microsoft.com/blog/azurehighperformancecomputingblog/optimizing-language-model-inference-on-azure/4248271, October 2024.

[14] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing LLM inference clusters for performance and energy efficiency, 2024.

[15] Qunyou Liu, Darong Huang, Marina Zapater, and David Atienza. Greenllm: Slo-aware dynamic frequency scaling for energy-efficient llm serving, 2025.

[16] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. MLPerf inference benchmark. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020. arXiv:1911.02549.

[17] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Sean Lai, Xupeng Miao, and Zhihao Jia. Adaserve: Slo-customized llm serving with fine-grained speculative decoding, 2025.

[18] MLCommons. MLPerf inference benchmarks documentation. https://docs.mlcommons.org/inference/, 2025.

[19] Vinh Nguyen, Wenwen Gao, Emily Apsey, Ganesh Kudleppanavar, Neelay Shah, and Elias Bermudez. LLM inference benchmarking: Fundamental concepts. https://developer.nvidia.com/blog/llm-benchmarking-fundamental-concepts/, April 2025.

[20] MLCommons. MLPerf inference v5.0 advances language model capabilities for GenAI. https://mlcommons.org/2025/04/llm-inference-v5/, April 2025.

[21] Vinh Nguyen, Wenwen Gao, Emily Apsey, Ganesh Kudleppanavar, Neelay Shah, and Elias Bermudez. LLM inference benchmarking: Fundamental concepts. https://developer.nvidia.com/blog/llm-benchmarking-fundamental-concepts/, April 2025.

[22] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[23] Mert Hidayetoglu, Aurick Qiao, Michael Wyatt, Jeff Rasley, Yuxiong He, and Samyam Rajbhandari. Shift parallelism: Low-latency, high-throughput llm inference for dynamic workloads. *arXiv preprint arXiv:2509.16495*, 2025.

[24] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.

[25] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[26] Shyamal Agarwal et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

[27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

[28] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[29] Qidong Su, Wei Zhao, Xin Li, Muralidhar Andoorveedu, Chenhao Jiang, Zhanda Zhu, Kevin Song, Christina Giannoula, and Gennady Pekhimenko. Seesaw: High-throughput llm inference via model re-sharding. *arXiv preprint arXiv:2503.06433*, 2025.

[30] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[31] vLLM Project. vLLM API Reference: vllm.model_executor.layers.linear (Source: vllm/model_executor/layers/linear.py). https://docs.vllm.ai/en/v0.10.1/api/vllm/model_executor/layers/linear.html.

[32] Bingyang Wu et al. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.

[33] PyTorch Contributors. Gloo: Collective communications library. https://github.com/pytorch/gloo, 2017.

[34] NVIDIA Corporation. NCCL: Nvidia collective communications library. https://github.com/NVIDIA/nccl, 2017.

[35] vLLM Project. vLLM v1: Next-Generation LLM Serving Engine. https://docs.vllm.ai/en/latest/. Released January 2025.
[36] Oak Ridge Leadership Computing Facility (OLCF). Defiant quick-start guide. https://docs.olcf.ornl.gov/ace_testbed/defiant_quick_start_guide.html.
[37] Aaron Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
[38] Chejian Xu, Wei Ping, Peng Xu, Zihan Liu, Boxin Wang, Mohammad Shoeybi, Bo Li, and Bryan Catanzaro. From 128k to 4m: Efficient training of ultra-long context large language models. *arXiv preprint arXiv:2504.06214*, 2025.
[39] Mert Hidayetoglu, Aurick Qiao, Michael Wyatt, Jeff Rasley, Yuxiong He, and Samyam Rajbhandari. Shift parallelism: Low-latency, high-throughput llm inference for dynamic workloads, September 2025.
[40] Krystal Hu et al. Chatgpt sets record for fastest-growing user base-analyst note. *Reuters*, 12:2023, 2023.
[41] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents, 2024.
[42] Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
[43] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.
[44] vLLM Project. Metrics. https://docs.vllm.ai/en/stable/design/metrics/.
[45] The Prometheus Authors. Prometheus - monitoring system & time series database. https://prometheus.io/, 2025.
[46] Mohammed Daffalla Elradi. Prometheus & grafana: A metrics-focused monitoring stack. *Journal of Computer Allied Intelligence (JCAI, ISSN: 2584-2676)*, 3(3):28–39, 2025.
[47] Zizhao Mo, Jianxiong Liao, Huanle Xu, Zhi Zhou, and Chengzhong Xu. Hetis: Serving llms in heterogeneous gpu clusters with fine-grained and dynamic parallelism. *arXiv preprint arXiv:2509.08309*, 2025.
[48] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 640–654, 2024.
[49] Haoyu Chen, Xue Li, Kun Qian, and Xin Wang. Gyges: Dynamic cross-instance parallelism transformation for efficient llm inference. *arXiv preprint arXiv:2509.19729*, 2025.
[50] Aixin Liu et al. Hap: Hybrid adaptive parallelism for efficient mixture-of-experts inference. *arXiv preprint arXiv:2508.19373*, 2025.
[51] Zhuohan Li et al. Alpaserve: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023.
[52] Marcel Wagenländer, Guo Li, Bo Zhao, Luo Mai, and Peter R Pietzuch. Tenplex: Dynamic parallelism for deep learning using parallelizable tensor collections. In *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '24)*, 2024.